Colonees

The General-Purpose Autonomous Agent Swarm Platform

Colonees is an open-source, production-grade platform that turns any combination of APIs, codebases, documents, and services into an intelligent, autonomous agent swarm. Connect your Swagger spec, point to your GitHub repo, upload your docs, wire up your services — and Colonees assembles a colony of specialist agents that can **understand, reason, and act** across all of them.

It is domain-agnostic and embeddable — deployable across customer support, finance, legal, healthcare, engineering, education, marketing, research, and any vertical where complex workflows need to run autonomously. Specialists are created, connected, and managed entirely through the API and web UI. No code required.

What You Can Build With Colonees

Application Copilot

> *"I want an AI copilot for my SaaS app"*

1. **Connectors** → Add "Custom API (OpenAPI/Swagger)" → paste `https://myapp.com/openapi.json`
   - System auto-parses every endpoint → generates callable tools (`create_user` , `get_order` , `list_tickets` , etc.)
   - Agents can now **perform real actions** through your app's API
2. **Connectors** → Add "GitHub Repository" → paste the frontend repo URL
   - System clones it, indexes all code → agents get `search_codebase` , `read_repo_file` , `get_repo_structure`
   - Agents can **understand the architecture**, UI components, and user journeys
3. **Knowledge Base** → Upload product docs, internal guides, FAQ articles
   - Agents get `search_knowledge_base` tool → they **know your product**
4. **Colonees** → Create specialists:

    - "App Navigator" (tools: repo + API) → understands UI flow and can explain how features work
    - "Action Agent" (tools: API) → performs actions on behalf of users
    - "Support Agent" (tools: KB + API) → answers questions using docs + live data
5. **Workspace** → Group all of the above into "MyApp Copilot" workspace
6. **Playground** → Select workspace → ask: "*Show me the last 5 orders for user john@example.com and explain the checkout flow*"
    - Supervisor spawns Action Agent (calls `get_orders` API) + App Navigator (searches codebase for checkout components)
    - Returns real data + architectural explanation

Customer Support Center

Connect your ticketing system's API, upload your support playbooks, wire up Slack — agents handle tickets, escalate issues, and respond to customers.

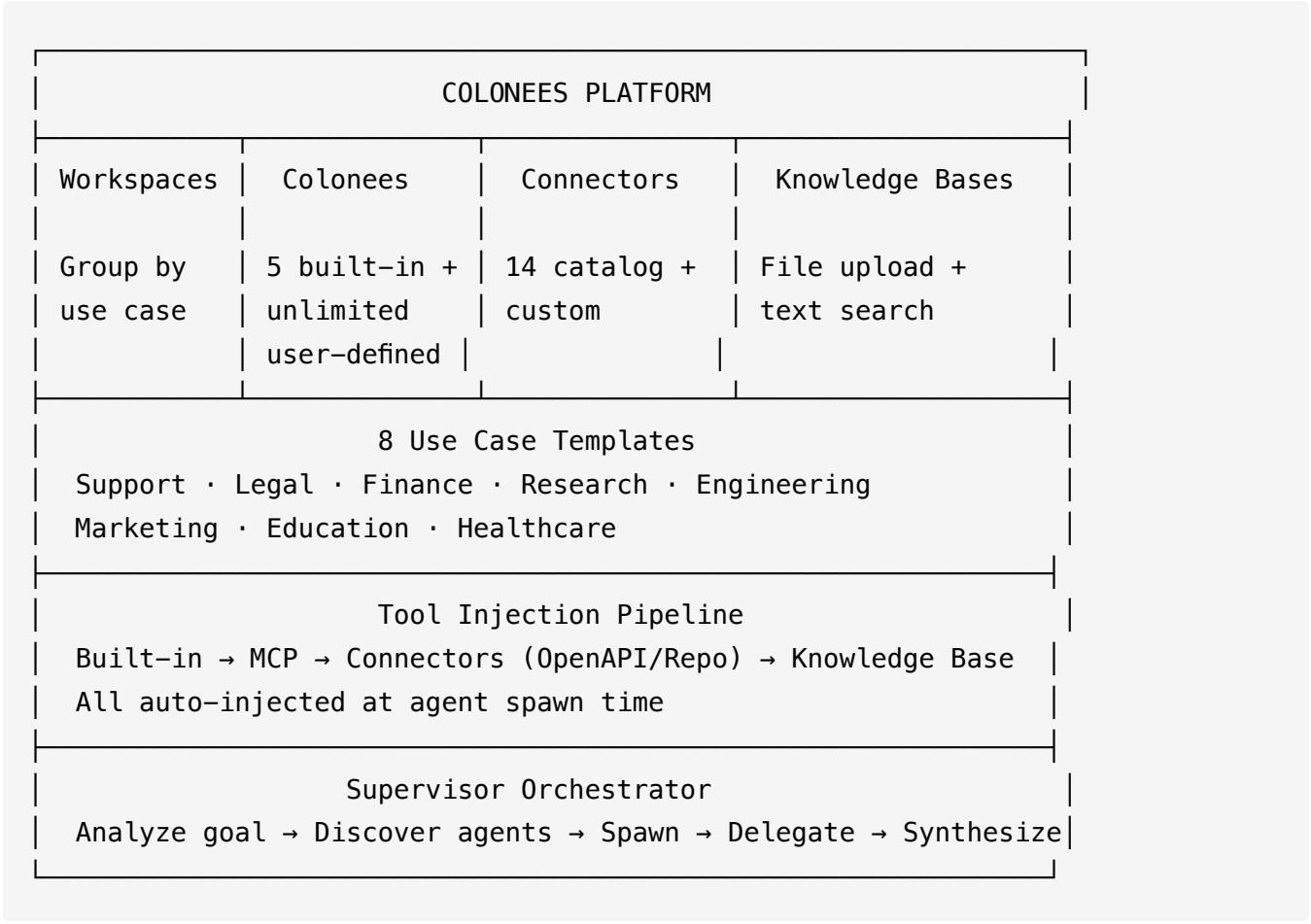
Legal Research Platform

Upload case law documents, connect legal databases, create contract analyzer and compliance checker agents that work together to review and annotate documents.

Internal Engineering Copilot

Point to your monorepo, connect your CI/CD API, upload your architecture docs — agents can explain code, investigate bugs, and even trigger deployments.

Platform Architecture



How It Works

1. **User submits a goal** (natural language) to the supervisor agent
2. **Supervisor analyzes** the goal and identifies required capabilities
3. **Discovers and spawns** the right specialist agents from the colony
4. **Each specialist** gets its tools injected automatically — built-in tools, MCP servers, connector-generated API tools, repo search tools, and knowledge base search
5. **Specialists execute** their tasks autonomously — calling APIs, searching code, querying knowledge bases
6. **Supervisor synthesizes** all results into a coherent response
7. **Session persists** — continue the conversation across requests

---

Quick Start

Prerequisites

- Python 3.10+
- Node.js 18+ (for the web UI)
- An LLM API key (Groq, OpenAI, Anthropic, or any OpenAI-compatible provider)

Installation

```
git clone https://github.com/colonees/colonees.git
cd colonees
pip install -e ".[dev]"
cp .env .env.local
# Edit .env.local — set LLM_API_KEY at minimum
```

Run the Backend

```
export LLM_API_KEY=your_key_here
python app.py
# API at http://localhost:8000
# Swagger docs at http://localhost:8000/docs
```

Run the Web UI

```
cd ui
npm install
npm run dev
# UI at http://localhost:5173
```

Docker

```
docker build -t colonees .
docker run -p 8000:8000 --env-file .env colonees
```

Web UI

Colonees ships with a full React dashboard for managing every aspect of the platform — no CLI or curl needed.

Page	What Users Do
Dashboard	Monitor live platform metrics, active sessions, agent spawns
Workspaces	Organize agents + connectors + knowledge bases by use case
Colonees	Create and configure specialist agent definitions
MCP Servers	Register low-level MCP tool server connections
Knowledge Base	Upload documents, search content, manage files
Connectors	Browse the service catalog, connect APIs and repos, auto-generate tools
Templates	One-click setup for 8 pre-built use cases
Sessions	View conversation history from past interactions
Playground	Chat with the agent swarm in real time
Settings	Platform configuration and API documentation links

Tech stack: Vite + React 18 + TypeScript + Tailwind CSS + TanStack Query + React Router

---

Connectors

Connectors are the integration layer that turns any external service into agent-usable tools. Instead of writing MCP servers or custom code, you **paste a URL or pick from the catalog** and Colonees generates tools automatically.

How Connectors Work

1. **Pick from the catalog** (or create a custom connector)
2. **Fill in credentials** (API key, token, etc.)
3. **Click "Connect"** — Colonees parses the spec, clones the repo, or establishes the connection
4. **Tools auto-generated** — each API endpoint becomes a callable tool, each repo becomes searchable
5. **Assign to workspace** — scope which agents can use which connectors

Connector Types

Type	What It Does	Tools Generated
OpenAPI / Swagger	Parses any OpenAPI spec	One tool per endpoint (GET, POST, PUT, DELETE, PATCH)
Git Repository	Clones and indexes a codebase	`search_codebase`, `read_repo_file`, `list_repo_files`, `get_repo_structure`
Database	Connects to PostgreSQL, MySQL, etc.	Query, schema inspection
Webhook	Sends data to any HTTP endpoint	Configurable HTTP call

Connector Catalog (14 Pre-Built)

Category	Connectors
API	Custom OpenAPI/Swagger
Code	GitHub, GitLab
Communication	Slack, Telegram
Google	Gmail, Google Docs, Google Drive

Category	Connectors
Database	PostgreSQL
Storage	AWS S3
Productivity	Notion, Linear, Jira
Integration	Custom Webhook

Example: Connect Your App's API

```
# 1. Create connector from the OpenAPI catalog entry
curl -X POST http://localhost:8000/connectors/from-catalog/openapi_custom

# 2. Update with your spec URL and credentials
curl -X PUT http://localhost:8000/connectors/openapi_custom \
  -H "Content-Type: application/json" \
  -d '{
    "config": {
      "spec_url": "https://api.myapp.com/openapi.json",
      "base_url": "https://api.myapp.com/v1",
      "auth_type": "bearer"
    },
    "credentials": {
      "api_key_or_token": "sk-your-api-key"
    }
  }'

# 3. Connect — parses the spec and generates tools
curl -X POST http://localhost:8000/connectors/openapi_custom/connect

# 4. See the auto-generated tools
curl http://localhost:8000/connectors/openapi_custom/tools
# → { "tools": ["list_users", "get_order", "create_ticket", ...], "count": 47 }
```

Example: Connect a GitHub Repository

```
# Create connector
curl -X POST http://localhost:8000/connectors \
  -H "Content-Type: application/json" \
  -d '{
    "name": "myapp_frontend",
    "display_name": "MyApp Frontend",
    "description": "Frontend React application",
    "type": "repo",
    "provider": "github",
    "config": {
      "repo_url": "https://github.com/myorg/myapp-frontend",
      "branch": "main"
    },
    "credentials": {
      "access_token": "ghp_your_token"
    }
  }'

# Connect — clones and indexes the repo
curl -X POST http://localhost:8000/connectors/myapp_frontend/connect
# → { "sync_stats": { "files_indexed": 342, "total_size_bytes": 2847291 } }
```

Workspaces

Workspaces group colonees, connectors, MCP servers, and knowledge bases into use-case bundles. When you invoke the swarm with a workspace, only the agents and tools in that workspace are available.

```
# Create a workspace
curl -X POST http://localhost:8000/workspaces \
  -H "Content-Type: application/json" \
  -d '{
    "name": "myapp_copilot",
    "display_name": "MyApp Copilot",
    "description": "AI copilot for the MyApp platform",
    "colonees": ["app_navigator", "action_agent", "support_agent"],
    "mcp_servers": [],
    "knowledge_bases": ["myapp_docs"],
    "icon": "bot",
    "color": "#3b82f6"
  }'

# Invoke with workspace context — auto-selects the right agents
curl -X POST http://localhost:8000/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "goal": "Show me recent failed orders and explain what might be causing them",
    "workspace": "myapp_copilot"
  }'
```

Knowledge Bases

Upload documents and files that agents can search and reference during task execution.

```
# Create a knowledge base
curl -X POST http://localhost:8000/knowledge-bases \
  -H "Content-Type: application/json" \
  -d '{
    "name": "product_docs",
    "display_name": "Product Documentation",
    "description": "All product documentation and guides",
    "type": "files"
  }'

# Upload files
curl -X POST http://localhost:8000/knowledge-bases/product_docs/upload \
  -F "file=@./docs/user-guide.md"
curl -X POST http://localhost:8000/knowledge-bases/product_docs/upload \
  -F "file=@./docs/api-reference.md"

# Search
curl -X POST http://localhost:8000/knowledge-bases/product_docs/search \
  -H "Content-Type: application/json" \
  -d '{"query": "how to reset password", "top_k": 5}'
```

Supported file types: `.txt`, `.md`, `.csv`, `.json`, `.yaml`, `.xml`, `.html`, `.log`

Templates

Pre-built workspace blueprints for common use cases. One click creates the workspace, colonees, and knowledge base placeholders.

Available Templates

Template	Agents Created	Category
Customer Support	FAQ Agent, Ticket Classifier, Escalation Handler, Feedback Analyzer	Support
Legal	Contract Analyzer, Compliance Checker, Legal	Legal

Template	Agents Created	Category
	Researcher, Case Summarizer	
Finance	Financial Analyst, Risk Assessor, Report Generator, Market Researcher	Finance
Research	Literature Reviewer, Data Analyst, Hypothesis Generator, Citation Manager	Research
Software Engineering	Code Reviewer, Bug Investigator, Documentation Writer, Test Generator, Architecture Advisor	Engineering
Marketing	Content Creator, SEO Analyst, Campaign Strategist, Audience Researcher	Marketing
Education	Tutor, Curriculum Designer, Quiz Generator, Progress Tracker	Education
Healthcare	Symptom Analyzer, Research Aggregator, Clinical Data Reviewer, Patient Communicator	Healthcare

```
# List templates
curl http://localhost:8000/templates

# Apply a template — creates workspace + colonees + KB placeholders
curl -X POST http://localhost:8000/templates/customer_support/apply
# → {
#     "workspace": "customer_support",
#     "colonees_created": ["faq_agent", "ticket_classifier", "escalation_handler", "feedba
#     "knowledge_bases_created": ["customer_support_kb"]
#   }
```

Colonees — Specialist Agent Definitions

Colonees are the specialist agent blueprints. Each defines a specialist's identity, tools, constraints, and evaluation criteria. Create them via the API or UI — no Python code required.

Built-in Colonees

Name	Type	Tools	Purpose
`researcher`	researcher	research, computation	Information gathering, synthesis, source validation
`domain_expert`	domain_expert	media, computation, research	Deep domain knowledge, explanations
`analyst`	analyst	file, media, computation	Data analysis, evaluation, structured reporting
`executor`	executor	file, media, computation	File operations, computation, task execution
`media_producer`	media_producer	media	Video, image, and audio generation

Create a Custom Colonee

```
curl -X POST http://localhost:8000/colonees \
  -H "Content-Type: application/json" \
  -d '{
    "name": "api_expert",
    "display_name": "API Expert",
    "description": "Calls application APIs to retrieve data and perform actions",
    "specialist_type": "executor",
    "system_prompt": "You are an API integration specialist for {specialization}. Use you
    "capabilities": ["api_calls", "data_retrieval", "crud_operations"],
    "built_in_tools": ["file", "research"],
    "mcp_servers": [],
    "tags": ["api", "integration"],
    "constraints": {
      "max_tool_calls": 30,
      "max_runtime_seconds": 300,
      "forbidden_topics": [],
      "output_format": "markdown"
    },
    "evaluation": {
      "quality_threshold": 0.7,
      "require_sources": false,
      "require_structured_output": false
    }
  }'
```

Colonee Fields

Field	Description
`name`	Unique slug — used to spawn the agent
`specialist_type`	Base type: `researcher`, `domain_expert`, `analyst`, `executor`, `media_producer`
`system_prompt`	Full system prompt. Use `{specialization}` as a placeholder
`built_in_tools`	Tool sets: `file`, `computation`, `research`, `media`

Field	Description
`mcp_servers`	Named MCP servers to attach
`capabilities`	Capability strings for automatic agent discovery
`constraints`	`max_tool_calls`, `max_runtime_seconds`, `forbidden_topics`, `output_format`
`memory`	`enabled`, `strategy` (`basic` / `semantic` / `episodic`), `max_size_mb`
`evaluation`	`quality_threshold`, `require_sources`, `require_structured_output`
`tags`	Free-form tags for filtering
`enabled`	Whether this colonee can be spawned (default `true`)

Per-Request Colonee Selection

```
# Only use specific colonees for this request
curl -X POST http://localhost:8000/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "goal": "Research and summarise the latest CRISPR breakthroughs",
    "colonees": ["researcher", "analyst"]
  }'

# Or use a workspace (auto-selects its colonees)
curl -X POST http://localhost:8000/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "goal": "Analyse Q3 sales",
    "workspace": "finance_team"
  }'
```

MCP Servers — Advanced Tool Integration

For users who need full control over tool servers, Colonees supports the Model Context Protocol (MCP) directly.

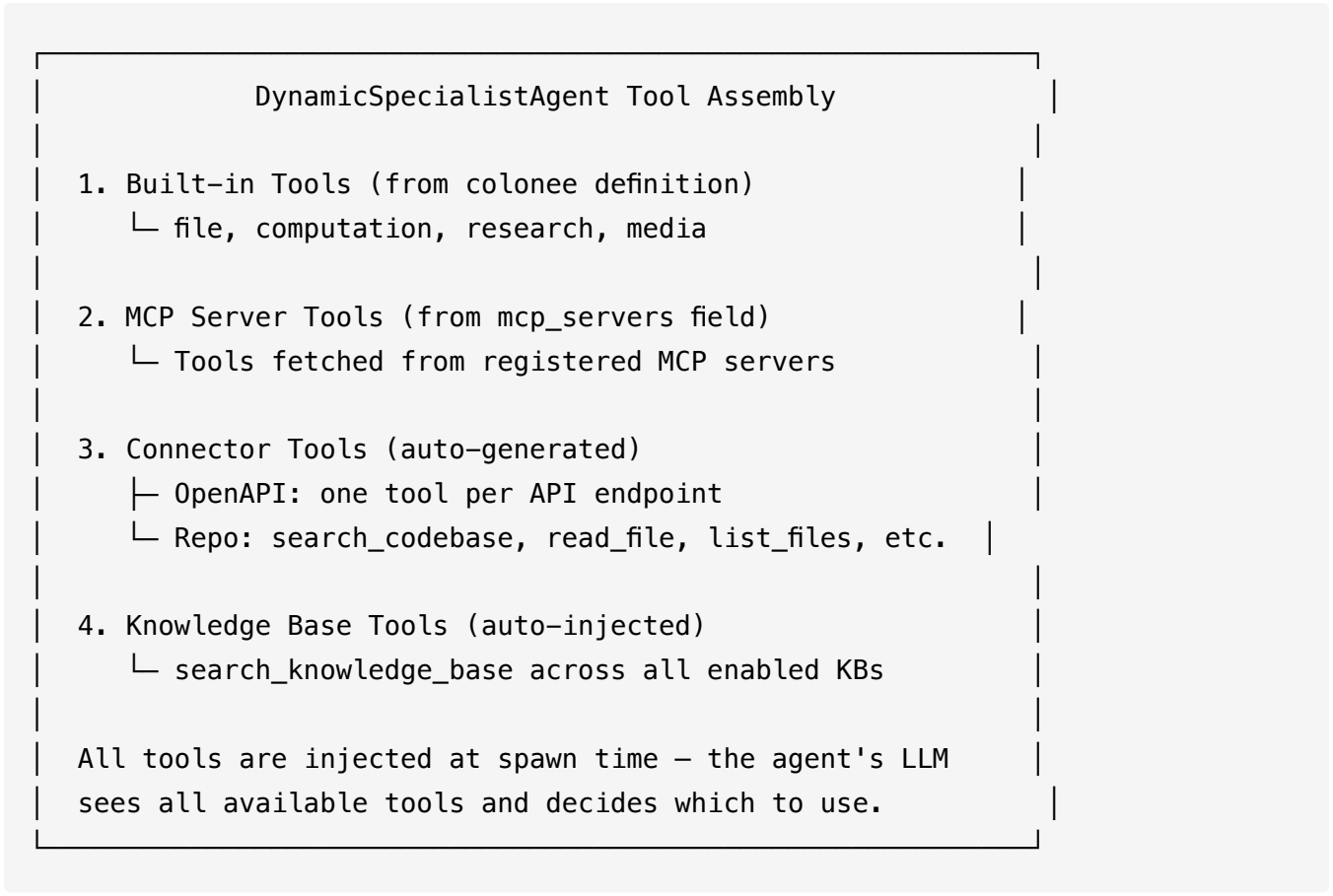
```
# HTTP server with API key auth
curl -X POST http://localhost:8000/mcp/servers \
  -H "Content-Type: application/json" \
  -d '{
    "name": "pubmed",
    "transport": "streamable_http",
    "url": "https://pubmed-mcp.example.com/mcp/",
    "headers": { "X-API-Key": "your-api-key-here" },
    "specialist_types": ["researcher"]
  }'

# stdio server with env vars
curl -X POST http://localhost:8000/mcp/servers \
  -H "Content-Type: application/json" \
  -d '{
    "name": "postgres",
    "transport": "stdio",
    "command": "python",
    "args": ["db_server.py"],
    "env": { "DB_PASSWORD": "secret" },
    "specialist_types": ["analyst"]
  }'
```

Supported transports: `streamable_http`, `sse`, `stdio`

Scoping: Use `specialist_types` to control which agents get which MCP tools. Empty = all agents.

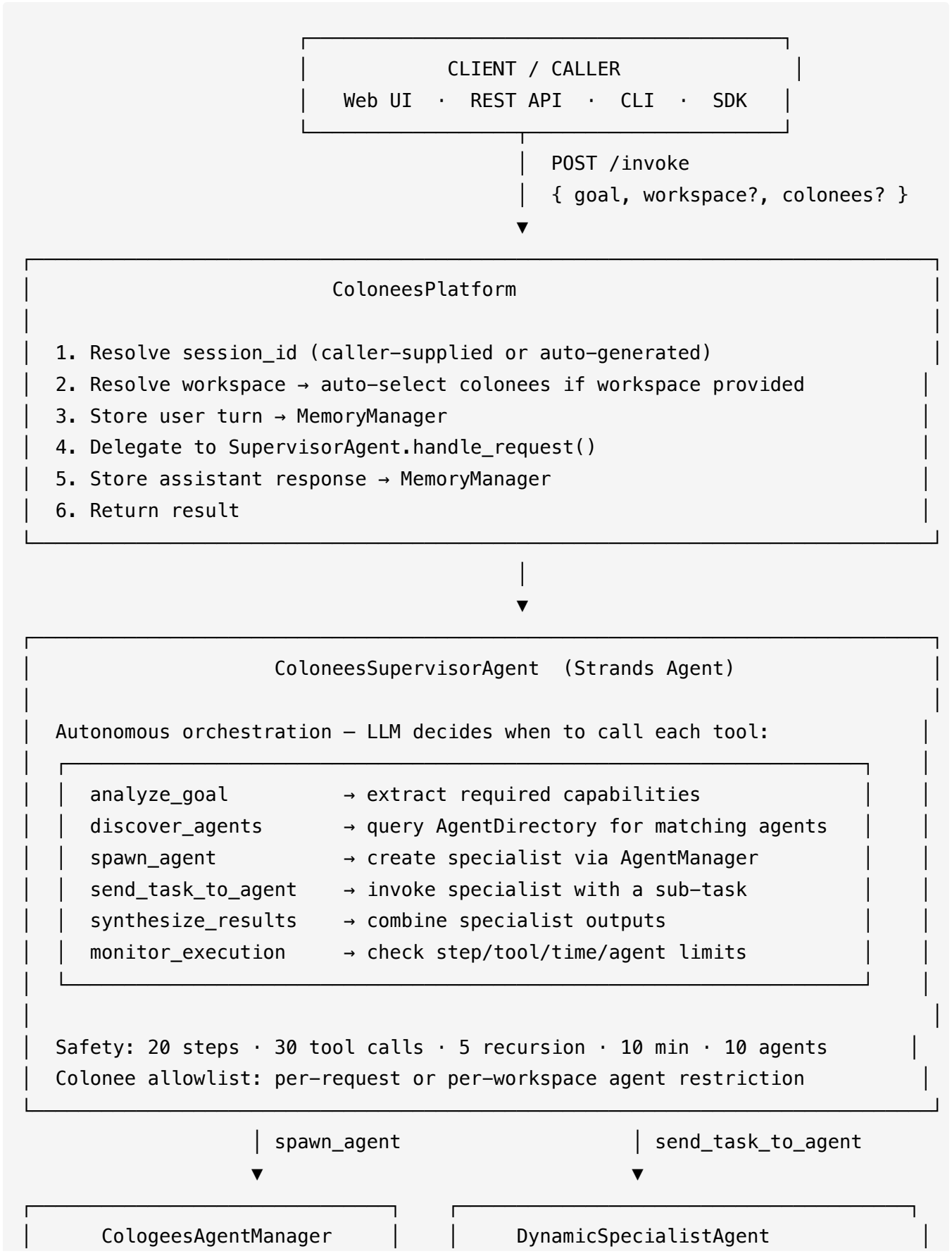
Tool Injection Pipeline

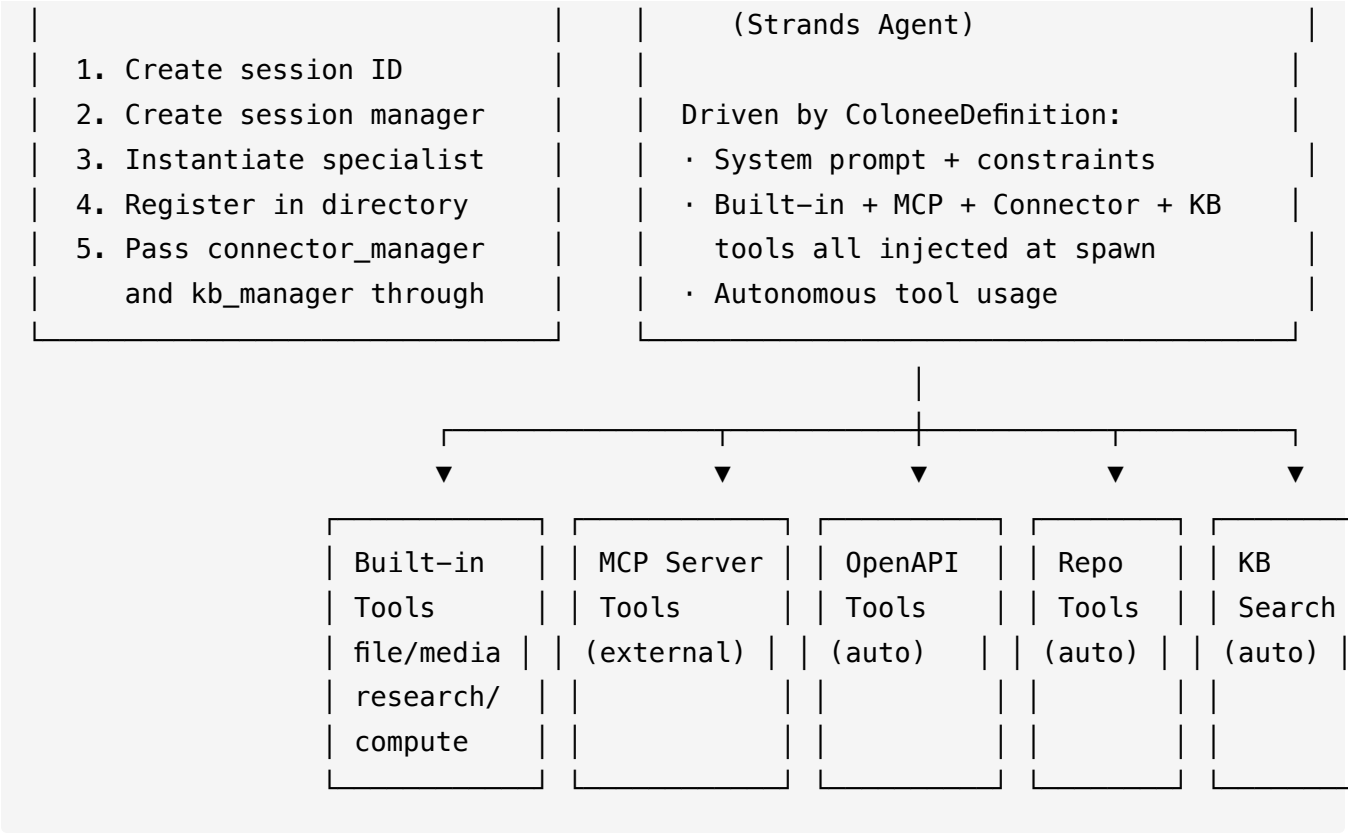
When a specialist agent is spawned, its tools are assembled automatically from four sources:


DynamicSpecialistAgent Tool Assembly
1. Built-in Tools (from colonee definition)
└─ file, computation, research, media
2. MCP Server Tools (from mcp_servers field)
└─ Tools fetched from registered MCP servers
3. Connector Tools (auto-generated)
├─ OpenAPI: one tool per API endpoint
└─ Repo: search_codebase, read_file, list_files, etc.
4. Knowledge Base Tools (auto-injected)
└─ search_knowledge_base across all enabled KBs
All tools are injected at spawn time — the agent's LLM
sees all available tools and decides which to use.

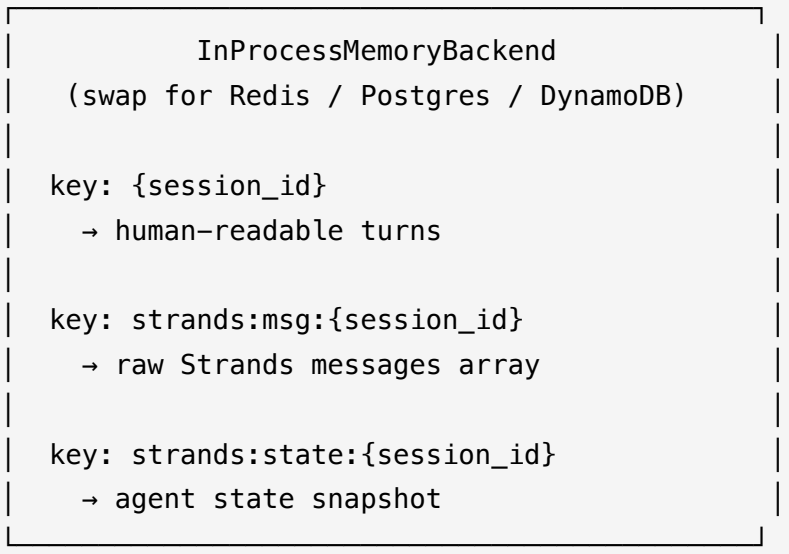
System Architecture

Full System Flow


CLIENT / CALLER
Web UI · REST API · CLI · SDK
POST /invoke
{ goal, workspace?, colonees? }
ColoneesPlatform
1. Resolve session_id (caller-supplied or auto-generated)
2. Resolve workspace → auto-select colonees if workspace provided
3. Store user turn → MemoryManager
4. Delegate to SupervisorAgent.handle_request()
5. Store assistant response → MemoryManager
6. Return result
ColoneesSupervisorAgent (Strands Agent)
Autonomous orchestration — LLM decides when to call each tool:
analyze_goal → extract required capabilities
discover_agents → query AgentDirectory for matching agents
spawn_agent → create specialist via AgentManager
send_task_to_agent → invoke specialist with a sub-task
synthesize_results → combine specialist outputs
monitor_execution → check step/tool/time/agent limits
Safety: 20 steps · 30 tool calls · 5 recursion · 10 min · 10 agents
Colonee allowlist: per-request or per-workspace agent restriction
spawn_agent
send_task_to_agent
CologeesAgentManager
DynamicSpecialistAgent



Memory & Session Architecture



Core Principles

- **Superagent orchestrates** — never executes tools directly

- **Specialists reason and use tools** — never orchestrate other agents
- **Tool providers execute** — no reasoning, no LLM calls
- **All specialist types are data-driven** (`ColoneeDefinition`) — no Python subclass needed
- **Connectors auto-generate tools** — paste a URL, get callable functions
- **One shared memory backend** for all history (human-readable + Strands LLM state)
- **Session IDs** are the single key tying memory, state, and lifecycle together

---

API Reference

System

Method	Endpoint	Description
GET	`/health`	Liveness probe
GET	`/status`	Platform status, metrics, configuration

Agent Swarm

Method	Endpoint	Description
POST	`/invoke`	Submit goal to the swarm (accepts `workspace`, `colonees`, `session_id`)
POST	`/agents/invoke`	Invoke a specific specialist directly

Colonees

Method	Endpoint	Description
POST	`/colonees`	Create specialist definition
GET	`/colonees`	List all (filter: `enabled_only`, `tag`)
GET	`/colonees/{name}`	Get single colonee

Method	Endpoint	Description
PUT	`/colonees/{name}`	Update colonee
DELETE	`/colonees/{name}`	Delete (user-defined only)
POST	`/colonees/{name}/enable`	Enable colonee
POST	`/colonees/{name}/disable`	Disable colonee

Connectors

Method	Endpoint	Description
GET	`/connectors/catalog`	Browse the connector catalog
GET	`/connectors/catalog/categories`	List catalog categories
POST	`/connectors/from-catalog/{id}`	Create connector from catalog template
POST	`/connectors`	Create custom connector
GET	`/connectors`	List connectors (filter: `type`, `workspace`)
GET	`/connectors/{name}`	Get connector (credentials masked)
PUT	`/connectors/{name}`	Update connector
DELETE	`/connectors/{name}`	Delete connector
POST	`/connectors/{name}/connect`	Activate — parse spec, clone repo
POST	`/connectors/{name}/sync`	Re-sync — re-pull repo, re-fetch spec
GET	`/connectors/{name}/tools`	List auto-generated tool names

Workspaces

Method	Endpoint	Description
POST	`/workspaces`	Create workspace
GET	`/workspaces`	List workspaces
GET	`/workspaces/{name}`	Get workspace
PUT	`/workspaces/{name}`	Update workspace
DELETE	`/workspaces/{name}`	Delete workspace
POST	`/workspaces/{name}/enable`	Enable workspace
POST	`/workspaces/{name}/disable`	Disable workspace

Knowledge Bases

Method	Endpoint	Description
POST	`/knowledge-bases`	Create knowledge base
GET	`/knowledge-bases`	List knowledge bases
GET	`/knowledge-bases/{name}`	Get knowledge base
PUT	`/knowledge-bases/{name}`	Update knowledge base
DELETE	`/knowledge-bases/{name}`	Delete knowledge base
POST	`/knowledge-bases/{name}/upload`	Upload file (multipart)
DELETE	`/knowledge-bases/{name}/files/{filename}`	Delete file
GET	`/knowledge-bases/{name}/files`	List files
POST	`/knowledge-bases/{name}/search`	Search knowledge base

Templates

Method	Endpoint	Description
GET	`/templates`	List templates (filter: `category`)
GET	`/templates/categories`	List template categories
GET	`/templates/{name}`	Get template detail
POST	`/templates/{name}/apply`	Apply template — creates workspace + colonees + KBs

MCP Servers

Method	Endpoint	Description
POST	`/mcp/servers`	Register MCP server
GET	`/mcp/servers`	List servers (headers masked)
DELETE	`/mcp/servers/{name}`	Remove server

Sessions & History

Method	Endpoint	Description
GET	`/history/{session_id}`	Get conversation history
DELETE	`/history/{session_id}`	Clear session history

Embedding Colonees

Use Colonees as a Python library in your own application:

```python
from colon.core.platform import ColoneesPlatform
from colon.core.config import CologeesConfig

config = CologeesConfig.from_environment()
platform = ColoneesPlatform(config)
await platform.initialize()

# Register connectors programmatically
platform.connector_manager.create(
    name="myapp_api",
    display_name="MyApp API",
    type="openapi",
    config={"spec_url": "https://api.myapp.com/openapi.json", "auth_type": "bearer"},
    credentials={"api_key_or_token": "sk-..."},
)

# Submit a goal
result = await platform.handle_request(
    user_goal="List all pending orders and summarize the backlog",
    context={"workspace": "myapp_copilot"}
)
```

CLI

```bash
# Submit a goal
colonees invoke --goal "Research the latest trends in quantum computing"

# With workspace context
colonees invoke --goal "Summarise Q3 sales" --workspace finance_team

# Platform status
colonees status
```

Environment Variables

Variable	Description	Default
`LLM_API_KEY`	LLM provider API key	required
`GEMINI_API_KEY`	Google Gemini API key (image gen + TTS)	required for media
`COLONEES_ENVIRONMENT`	`local` or `production`	`local`
`COLONEES_LOG_LEVEL`	Log level	`INFO`
`COLONEES_FILES_DIR`	Local directory for file storage	`colonees_files`
`COLONEES_MEDIA_DIR`	Local directory for media productions	`colonees_media`
`COLONEES_STORAGE_DIR`	Storage for workspaces, KBs, connectors	`colonees_files`
`COLONEES_MAX_USERS`	Max concurrent users	`100000`
`COLONEES_MAX_SESSIONS`	Max active sessions	`10000`
`PORT`	HTTP server port	`8000`

Safety

- Max 20 agent steps per task
- Max 30 tool calls per task
- Max 5 recursion depth
- Max 10 minutes runtime per task
- Max 10 agents per task
- Goal validation against forbidden patterns
- Per-colonee `forbidden_topics` enforced via system prompt
- Connector credentials are never exposed in API responses

Project Structure

```
colonees/
├── app.py                          # FastAPI entrypoint (all API routes)
├── requirements.txt
├── Dockerfile
│
├── colon/                          # Core platform package
│   ├── core/
│   │   ├── platform.py             # ColoneesPlatform orchestrator
│   │   ├── config.py               # Platform configuration
│   │   ├── workflow_orchestrator.py # Supervisor agent
│   │   ├── workspace.py            # Workspace management
│   │   ├── knowledge_base.py       # Knowledge base management
│   │   ├── templates.py            # Use case templates
│   │   └── resource_manager.py     # Resource monitoring
│   │
│   ├── agents/
│   │   ├── colonee_registry.py     # Colonee definitions + persistence
│   │   ├── agent_manager.py        # Agent spawning + lifecycle
│   │   ├── dynamic_specialist.py   # Tool injection + agent creation
│   │   ├── specialist_agents.py    # Agent factory
│   │   ├── agent_directory.py      # Capability-based discovery
│   │   └── tools/                  # Built-in tool providers
│   │       ├── file/               # File operations (15 tools)
│   │       ├── media/              # Image, video, audio generation
│   │       ├── research/           # Web search, URL fetch
│   │       ├── computation/        # Python exec, math eval
│   │       └── knowledge/          # Knowledge base search
│   │
│   ├── connectors/                 # External service integrations
│   │   ├── connector_registry.py   # Connector definitions + persistence
│   │   ├── openapi_connector.py    # Swagger/OpenAPI → auto-generated tools
│   │   ├── repo_connector.py       # Git clone → code search tools
│   │   └── connector_catalog.py    # 14 pre-built connector templates
│   │
│   ├── memory/                     # Session and conversation persistence
│   │   ├── memory_manager.py       # Conversation history
│   │   ├── session_manager.py      # Session lifecycle
│   │   └── strands_session.py      # Strands LLM state persistence
│   │
```

```
│   └── communication/
│       └── mcp_manager.py          # MCP server connections
│
└── ui/                             # React web dashboard
    ├── package.json
    └── src/
        ├── api/                    # API clients (axios)
        ├── hooks/                  # React Query hooks
        ├── components/             # UI components (shadcn-style)
        └── pages/                  # 10 page components
```

License

MIT License — see LICENSE for details.

Links

- GitHub: https://github.com/colonees/colonees
- Issues: https://github.com/colonees/colonees/issues
- Email: alaminibrahim433@gmail.com